

OpenDigger Top300项目数据获取

一、获取Top300项目信息

1.1 信息内容

- 项目名称：repo_name
- 项目所有者：company
- 项目排名：rank

获取地址：https://oss.x-lab.info/open_leaderboard/open_rank/repo/global/2025.json

注：该地址提供了2025年排名前300项目的json数据信息。

二、获取每个项目的各指标数据

2.1 指标列表（14个）

指标	聚合指标	类型
issues_new	项目活跃度、开发者活跃度、Github指数	分类
issues_closed	Github指数	分类
issue_comments	项目活跃度、开发者活跃度	分类
issue_resolution_duration	Github指数	时间
change_requests	项目活跃度、开发者活跃度、PREI、Github指数	分类
change_requests_reviews	项目活跃度、开发者活跃度、PREI	分类
change_requests_accepted	PREI、Github指数	分类
change_request_response_time	PREI	时间
change_request_resolution_duration	PREI、Github指数	时间
stars	关注度、Github指数	分类
technical_fork	关注度、Github指数	分类

指标	聚合指标	类型
new_contributors	开发者活跃度、Github指数	分类
openrank	OpenRank	分类
activity	activity(用于排序)	分类

指标	类别	备注
项目活跃度	月度数据	
开发者活跃度	月度数据	
关注度	月度数据	
PREI	月度数据	
Github指数	总体数据	
OpenRank	月度数据	

月度数据：该聚合指标按月展示；总体数据：该聚合指标是一个综合指标，每个项目一个。

2.2 获取指标

2.2.1 拼接指标下载地址

```
url=https://oss.open-digger.cn/github/{company}/{project}/{metric}.json
```

将指标数据文件保存到/top300_metric/{company}/{project}/{metric}.json

文件结构重组

从/top300_metric/{company}/{project}/{metric}.json转
为/top300_metric/{metric}/{company}_{project}.json

三、裁剪数据

3.1 分类数据裁剪策略

只保留YYYY-MM：value，删除年份、季度、raw以及超日期数据（只保留2021-01——2025-10）

3.2 时间数据裁剪策略

只保留YYYY-MM：value，删除年份、季度、超日期以及所有分位结构等数据

四、数据合并

将分类数据和时间数据合并到一张表中，表字段：company、project、metric、date、value（在月份数据不缺失的情况下，相当于每个项目有58（月数）*14（指标数）=812条记录）

检查每个项目的月份是否有缺失，进行缺失值处理：

1、如果某个项目2021-01——2021-08数据缺失即某个项目是在2021-01之后发布到Github，前面月份数据使用0填充。判断一个项目是否是2021-01之后发布的，检查2021-01——起始月份之间是没有数据的，这样的话是符合的；即从有数据的月份开始向前填充0；

2、如果2021-01有数据，然后到2025-10之间缺失月份，缺失月份使用前一个月份的数据填充。

最后得到一张不缺月份完整的一张表。

五、聚合指标计算

5.1 项目活跃度

项目活跃度 = 创造活动 + 参与活动

创造活动 = $0.4 * \text{issues_new} + 0.4 * \text{change_requests}$

参与活动 = $0.1 * \text{issue_comments} + 0.1 * \text{change_requests_reviews}$

项目活跃度 = $0.4 * \text{issues_new} + 0.4 * \text{change_requests} + 0.1 * \text{issue_comments} + 0.1 * \text{change_requests_reviews}$

5.2 开发者活跃度

开发者活跃度 = $0.5 * \text{new_contributors} + 0.3 * (\text{issue_comments} + \text{change_requests_reviews}) + 0.2 * (\text{issues_new} + \text{change_requests})$

5.3 关注度

关注度 = $0.4 * \text{stars} + 0.6 * \text{technical_fork}$

5.4 PREI(PR处理效率指数)

$$PREI_{raw} = 0.35R_{resp} + 0.35R_{res} + 0.15R_{review} + 0.15R_{accept}$$

四个维度公式如下：

① 响应效率 (PR Response Efficiency)

$$R_{resp} = \text{MinMaxNorm}(\text{StdNorm}(1 - \log(1 + T_{resp})))$$

② 处理效率 (PR Resolution Efficiency)

$$R_{res} = \text{MinMaxNorm}(\text{StdNorm}(1 - \log(1 + T_{res})))$$

③ 审阅充分度 (Review Intensity)

$$ReviewIntensity = \frac{PR_Reviews}{PR_Count}$$

$$R_{review} = \text{MinMaxNorm}(\text{StdNorm}(ReviewIntensity))$$

④ PR 接受率 (PR Acceptance Rate)

$$AcceptRate = \frac{PR_Accepted}{PR_Count}$$

$$R_{accept} = \text{MinMaxNorm}(\text{StdNorm}(AcceptRate))$$

按照PREI公式加权求得综合得分，再对综合得分归一化处理后做功效系数。

$$PREI = 60 + 40 \times \text{MinMaxNorm}(PREI_{raw})$$

把四个维度的值也做功效系数用于展示。

5.5 Github指数

Github指数 = 0.3 * 影响力 + 0.2 * 社区反应 + 0.2 * 开发活跃度 + 0.3 * 发展趋势

影响力 = 0.25 * stars_sum + 0.25 * technical_fork_sum + 0.3 * issues_new_sum + 0.2 * change_requests_sum

社区反应 = 0.5 * issues_closed_sum + 0.2 * change_requests_accepted_sum + 0.2 * (1 - min_max_norm(issue_resolution_duration_sum)) + 0.1 * (1 - min_max_norm(change_request_resolution_duration_sum))

开发活跃度 = 0.4 * issues_new_sum + 0.3 * change_requests_sum + 0.3 * new_contributors_sum

发展趋势 = 0.4 * Issue 增长率 + 0.4 * PR 增长率 + 0.2 * 新增贡献者增长率，其中三个增长率均按月度环比变化率平均值 计算。

① Issue 增长率公式

$$\text{IssueTrend} = \text{mean} \left(\frac{Issues_t - Issues_{t-1}}{Issues_{t-1}} \right)$$

② PR 增长率公式

$$\text{PRTrend} = \text{mean} \left(\frac{PR_t - PR_{t-1}}{PR_{t-1}} \right)$$

③ 新增贡献者增长率公式

$$\text{DevTrend} = \text{mean} \left(\frac{NewContrib_t - NewContrib_{t-1}}{NewContrib_{t-1}} \right)$$

数据处理：

1. 对四个维度的原始项目级指标做 Min-Max 归一化（项目间 0~1 统一尺度）；
2. 对归一化后的四个维度做平方根平滑（降低极端值影响）；
3. 使用平滑后的四个维度按权重计算综合得分 Github_raw（理论上在 0~1 内，但实际区间较窄）；
4. 在所有项目上对 Github_raw 再做一次 Min-Max 归一化，得到 Github_norm，并对 Github_norm 做功效系数：
Github_Index = 60 + 40 * Github_norm（最终范围 60~100）；
5. 为了展示效果，对四个维度的平滑结果分别再做一次 Min-Max 归一化，并做功效系数：
Dimension_Index = 60 + 40 * Min-Max(平滑维度值)，仅用于展示，不影响第 3-4 步的综合得分计算逻辑。

为了后续增加的项目计算Github指数以及PREI，处理的时候要存储归一化时的min和max。